

当代数据库大作业一

课程名称:	当代数据管理系统	日期:	2025.10.31
姓名:	孙嘉	学号:	10235501445
姓名:	罗文琦	学号:	10235501417
年级:	2023 级	主题:	Bookstore

1 实验要求

本项目要求实现一个支持网上购书功能的网站后端，买家和卖家都可以注册账号。卖家可创建一个或多个商店、上架书籍并管理库存；买家可充值、下单、付款并完成完整的下单 → 付款 → 发货 → 收货交易流程。

项目主要包括两部分任务：

1. 基础功能（60%）

- 用户权限接口：注册、登录、登出、注销；
- 买家接口：充值、下单、付款；
- 卖家接口：创建店铺、添加书籍信息、增加库存；
- 所有接口需通过提供的测试用例。

2. 扩展功能（40%）

- 完善后续流程（发货、收货）；
- 实现搜索功能：支持按题目、标签、目录、内容搜索，全站或店铺内搜索，并实现分页与索引优化；
- 实现订单管理：支持订单查询、取消、超时自动取消等。

此外，需将项目迁移至**本地 MongoDB 数据库**，并对所有接口编写测试代码，提升测试覆盖率与性能表现。项目不要求前端界面，但需通过代码测试验证功能正确性，并使用 **Git** 进行版本管理。

2 数据库结构

根据书店数据的业务逻辑，数据可以基本划分为 user,store,book,order，我们发现在 order 中只有在查询历史订单信息的时候我们才需要知道 order 具体内容是什么，在支付、收书、发书等其他订单场景下，我们并不需要那么多信息。因此，我们将 order 中的数据分为两个文档集：order 和 order_history。针对 user_store 表，我们认为 store 文档集中的一个文档可以满足存储 user_id 的需求，因为一个 store 一定只被一个用户拥有，但是一个用户可以有很多商店。这样做可以满足即不增加多余内容，又满足实际情况。**最终设计成果：**

- **user**: 存储用户基础信息;
- **store**: 包含商店信息及 **user_id** 引用;
- **book**: 存储图书信息;
- **order**: 记录订单的基本信息 (如订单号、时间、状态等);
- **order_history**: 存储订单中图书的详细内容 (如书目、数量、价格等)。

这种设计兼顾了数据的**查询效率**、**结构清晰性**与**模型灵活性**, 充分体现了文档数据库在处理层次化业务关系中的优势, 具体结构如下:

```
1 user {
2     user_id,
3     password,
4     balance,
5     token,
6     terminal
7 }
8
9 store {
10     store_id,
11     user_id,
12     books[] {
13         book_id,
14         stock_level
15     }
16 }
17
18 book {
19     id,
20     title,
21     author,
22     ...,
23     content,
24     tags,
25     picture
26 }
27
28 order {
29     order_id,
30     store_id,
31     user_id,
32     status,
33     price
```

```

34 }
35
36 order_history {
37     order_id,
38     book_id,
39     count,
40     price
41 }

```

补充：status 中，0: 未付款; 1: 已付款未发货; 2: 已发货未接收; 3: 已接收; 4: 已取消。

3 索引

采用索引对于“频繁查找、不常更改”的数据项来说，在查询性能上具有显著的提升作用。结合整体实验设计并考虑各文档集的实际使用情况，我们最终设置了如下索引方案：

1. 在 `store` 文档集的 `store_id` 字段上设置升序索引，并保证其唯一性；
2. 在 `user` 文档集的 `user_id` 字段上设置升序索引，并保证其唯一性；
3. 为了支持对文本字段的全文搜索，在 `books` 文档集上设置了多个索引，分别作用于 `title`、`tags`、`book_intro` 与 `content` 字段（这些字段内容均为文本类型，因此索引类型均设置为 `text`）。

由于 `order` 与 `order_history` 这两个文档集的数据更新频繁，虽然其查询场景较为常见，但为避免索引维护带来的性能开销，我们最终放弃了在此类文档集上建立索引。

4 基本功能

4.1 用户模块

函数名	register（用户注册）
功能说明	注册新用户，生成唯一终端标识和令牌。
参数	user_id: 用户 ID; password: 用户密码。
返回值	状态码（200 成功，528 错误）； 消息字符串（"ok" 或错误提示）。
主要流程	1. 生成终端标识 terminal_<timestamp>。 2. 生成 token。 3. 插入用户信息到数据库。 4. 异常时返回 528 + 错误信息，否则返回 200 + "ok"。

函数名	unregister (用户注销)
功能说明	删除用户账号信息。
参数	user_id: 用户 ID; password: 用户密码。
返回值	状态码 (200 成功, 530 异常); 消息字符串 ("ok" 或错误提示)。
主要流程	1. 调用 check_password 验证密码。 2. 密码错误返回错误信息。 3. 删除数据库中匹配用户。 4. 成功返回 200 + "ok", 失败或异常返回错误消息。

函数名	check_token (验证令牌)
功能说明	验证用户提供的令牌是否正确。
参数	user_id: 用户 ID; token: 令牌。
返回值	状态码 (200 成功, 528 异常); 消息字符串 ("ok" 或错误提示)。
主要流程	1. 根据 user_id 查询数据库。 2. 不存在则返回授权失败。 3. 调用 __check_token 比对。 4. 匹配成功返回 200 + "ok", 否则返回错误信息。

函数名	check_password (验证密码)
功能说明	验证用户密码是否正确。
参数	user_id: 用户 ID; password: 密码。
返回值	状态码 (200 成功, 528 异常); 消息字符串 ("ok" 或错误提示)。
主要流程	1. 查询用户信息。 2. 比对密码是否一致。 3. 成功返回 200 + "ok", 失败返回授权错误。

函数名	login (用户登录)
功能说明	验证身份并生成新的令牌。
参数	user_id: 用户 ID; password: 密码; terminal: 终端标识。
返回值	状态码 (200 成功, 528 异常); 消息字符串; 令牌字符串 (成功时返回)。
主要流程	1. 调用 check_password 验证。 2. 成功后生成新 token 并更新数据库。 3. 异常返回 528, 成功返回 200 + "ok" + token。

函数名	logout (用户登出)
功能说明	退出登录, 使当前 token 失效。
参数	user_id: 用户 ID; token: 当前令牌。
返回值	状态码 (200 成功, 528 异常); 消息字符串。
主要流程	1. 调用 check_token 验证令牌。 2. 生成虚拟 token 和 terminal。 3. 更新数据库, 置空旧 token。 4. 成功返回 200 + "ok", 失败返回错误消息。

函数名	change_password (修改密码)
功能说明	修改用户密码并刷新 token。
参数	user_id: 用户 ID; old_password: 旧密码; new_password: 新密码。
返回值	状态码 (200 成功, 528 异常); 消息字符串。
主要流程	1. 调用 check_password 验证旧密码。 2. 成功后生成新 token 与 terminal。 3. 更新数据库中的密码和令牌。 4. 成功返回 200 + "ok", 异常返回 528 + 错误描述。

以上函数构成了完整的用户生命周期管理逻辑。总体上,所有函数在出错时均返回错误码 (528 或 530),便于前端捕获。核心操作包括 token 管理、密码校验与数据库同步更新。

4.2 卖家模块

5 卖家模块

函数名	add_new_book (添加书籍)
功能说明	卖家向指定商店中添加新的书籍信息。
参数	user_id: 执行操作的用户 ID; store_id: 目标商店 ID; book_id: 要添加的书籍 ID; book_json_str: 书籍信息的 JSON 字符串; stock_level: 库存数量。
返回值	状态码 (200 成功, 528 错误); 消息字符串 ("ok" 或错误提示)。
主要流程	1. 检查用户是否存在, 不存在则返回错误。 2. 检查商店是否存在, 不存在则返回错误。 3. 检查书籍是否已存在于商店, 存在则返回“书籍已存在”错误。 4. 插入书籍信息到商店集合 (含库存信息)。 5. 同步插入书籍详细信息到书籍集合。 6. 若出现异常, 返回状态码 528 与异常说明。

函数名	create_store (创建商铺)
功能说明	用户创建新的商店并与其账户绑定。
参数	user_id: 执行操作的用户 ID; store_id: 要创建的商店 ID。
返回值	状态码 (200 成功, 528 错误); 消息字符串 ("ok" 或错误提示)。
主要流程	1. 检查用户是否存在, 若不存在返回错误。 2. 检查商店 ID 是否已存在, 若已存在返回“商店已存在”错误。 3. 调用 insert_one 将新商店信息插入 store_col 集合, 建立用户与商店的关联。 4. 若发生异常, 捕获并返回 528 与错误信息。

函数名	add_stock_level (添加书籍库存)
功能说明	在商店中为已存在的书籍增加库存数量。
参数	user_id: 执行操作的用户 ID; store_id: 商店 ID; book_id: 书籍 ID; add_stock_level: 增加的库存数量。
返回值	状态码 (200 成功, 528 错误); 消息字符串 ("ok" 或错误提示)。
主要流程	1. 检查用户是否存在, 不存在则返回错误。 2. 检查商店是否存在, 不存在则返回错误。 3. 检查书籍是否存在于商店, 若不存在返回“书籍不存在”错误。 4. 使用 update_one 更新库存, 增加 add_stock_level 数量。 5. 若出现异常, 捕获并返回 528 与错误信息。

总结: 卖家模块接口主要处理商铺创建与书籍管理逻辑, 包括书籍信息新增和库存更新。所有接口在遇到异常时统一返回状态码 528, 便于前端统一处理错误反馈。

6 买家模块 API 文档总结 (表格版)

本节以表格形式总结买家模块的主要接口, 包括订单创建、支付及资金管理等操作, 语言简明自然, 方便理解与查阅。

函数名	new_order (创建新订单)
功能说明	用户在指定商店中创建一个新的订单。
参数	user_id : 用户 ID; store_id : 商店 ID; id_and_count : 包含书籍 ID 与数量的列表 (每项为元组)。
返回值	状态码 (200 成功, 528 错误); 消息字符串 ("ok" 或错误提示); 订单 ID 字符串。
主要流程	1. 初始化空字符串 order_id。 2. 检查用户与商店是否存在, 不存在则返回错误与空订单 ID。 3. 生成唯一订单 ID (包含用户 ID、商店 ID 与时间标识)。 4. 遍历 id_and_count: • 检查书籍是否存在, 不存在则返回错误。 • 检查库存是否足够, 不足则返回库存不足错误。 • 若库存充足, 更新库存并插入订单详情。 5. 计算总价并记录创建时间, 将订单插入订单集合。 6. 成功返回 (200, "ok", 订单 ID)。 7. 捕获异常并返回 (528, 异常信息, 空订单 ID)。

函数名	payment (支付)
功能说明	用户为已创建的订单进行支付操作。
参数	user_id: 用户 ID; password: 用户密码; order_id: 订单 ID。
返回值	状态码 (200 成功, 528 错误); 消息字符串 ("ok" 或错误提示)。
主要流程	1. 查询订单集合中状态为 0 (未付款) 的订单, 若不存在返回“无效订单 ID”。 2. 提取买家 ID、商店 ID、总价。 3. 验证用户是否为订单买家, 不匹配返回授权失败。 4. 验证用户是否存在且密码正确, 否则返回授权失败。 5. 检查商店是否存在, 不存在返回错误。 6. 检查卖家是否存在, 不存在返回错误。 7. 检查用户余额是否足够, 若不足返回“余额不足”。 8. 从买家账户中扣除订单金额。 9. 将订单金额添加到卖家账户中。 10. 更新订单状态为 1 (已支付), 插入支付记录。 11. 删除原状态为 0 的订单, 若失败返回“无效订单 ID”。 12. 捕获异常并返回状态码 528 与错误信息。

函数名	add_funds (添加资金)
功能说明	用户向自己的账户余额中添加资金。
参数	user_id: 用户 ID; password: 用户密码; add_value: 要添加的金额。
返回值	状态码 (200 成功, 528 错误); 消息字符串 ("ok" 或错误提示)。
主要流程	1. 根据 user_id 查找用户信息, 若不存在返回授权失败。 2. 验证输入密码是否正确, 不匹配返回授权失败。 3. 使用 \$inc 操作符增加用户账户余额。 4. 若未匹配到用户, 返回“用户不存在”。 5. 若出现异常, 捕获并返回状态码 528 与异常信息。

买家模块主要处理订单生成、支付与资金管理逻辑。所有函数均在出现异常时统一返回状态码 528，以便前端进行一致的错误处理与反馈。

7 拓展功能

7.1 发货与收货

7.1.1 卖家发货 (send_books)

函数定义于 `be/model/seller.py`，用于卖家将订单标记为已发货。

函数接受以下参数：

- `user_id`: 执行发货操作的用户 ID。
- `order_id`: 需要标记为已发货的订单 ID。

函数返回一个元组，包含两个值：

- 一个整数：操作状态码。200 表示成功，528 表示异常或错误。
- 一个字符串：描述操作结果，如“发货成功”或错误提示信息。

函数主要执行流程如下：

1. 查询订单，条件为订单 ID 等于 `order_id`，且订单状态为 1、2 或 3，表示订单已支付或已发货但尚未完成。
2. 若未找到符合条件的订单，返回“无效订单 ID”的错误提示。
3. 获取订单对应的商店 ID 及支付状态。
4. 验证操作用户是否为商店所有者，若不是，返回授权失败的错误消息。
5. 检查订单支付状态是否为 2 或 3，若是，返回“重复发货”的提示。
6. 使用 `update_one` 方法将订单状态更新为 2，表示已发货。
7. 在执行过程中若发生异常，捕获异常并返回状态码 528 及异常描述。

7.1.2 买家收货 (receive_books)

函数定义于 `be/model/buyer.py`，用于买家确认订单收货。

函数接受以下参数：

- `user_id`: 买家用户 ID。
- `order_id`: 需要确认收货的订单 ID。

函数返回一个元组，包含两个值：

- 一个整数：操作状态码。200 表示成功，528 表示异常或错误。
- 一个字符串：描述操作结果，如“收货成功”或错误提示信息。

函数主要执行流程如下：

1. 在订单集合中查找订单，条件为订单 ID 等于 *order_id* 且状态为 1、2 或 3，使用 \$or 操作符匹配。
2. 若未找到符合条件的订单，返回“无效订单 ID”的错误提示。
3. 提取订单中的买家 ID 和支付状态。
4. 验证操作用户 ID 是否与订单买家 ID 一致，若不一致，返回授权失败的错误消息。
5. 根据订单支付状态判断是否允许收货：状态为 1 表示未发货，返回“书籍未发出”的错误；状态为 3 表示已收货，返回“重复收货”的提示。
6. 使用 *update_one* 方法将订单状态更新为 3，表示订单已被接收。
7. 在执行过程中若发生异常，捕获异常并返回状态码 528 及异常描述。

7.2 搜索图书（包括优化）

7.2.1 简单搜索（初版 search）

最初在 *buyer* 模块中实现了一个基础的 *search* 方法，用于根据关键字查找书籍。

函数接受以下参数：

- *keyword*：搜索关键字。
- *store_id*：商店 ID，用于限定搜索范围，可选。
- *page*：页码，分页显示结果，默认值为 1。
- *per_page*：每页显示书籍数量，默认值为 10。

函数返回一个元组：

- 一个整数：操作状态码，200 表示成功，530 表示异常或错误。
- 一个列表：包含查询到的书籍信息。

函数主要执行流程：

1. 构建基础查询 *base_query*，使用 MongoDB 的 \$text 操作符对 *keyword* 执行全文搜索。
2. 初始化查询条件 *query* 为 *base_query*。
3. 如果提供了 *store_id*：

- (a) 在商店集合中查找对应商店的书籍 ID，保存到列表 `books_id`。
- (b) 在查询条件中添加书籍 ID 必须属于 `books_id` 的限制。
4. 执行查询，检索符合条件的书籍，同时获取每本书的文本相关性分数 (`$meta`)。
5. 执行分页操作，跳过前面 $(page - 1) * per_page$ 条结果，并限制每页显示 `per_page` 条。
6. 若执行过程中发生异常，捕获异常并返回状态码 530 以及异常描述。
7. 返回状态码 200 和包含搜索结果的书籍列表。

7.2.2 优化说明

经过实践发现初版设计存在以下问题：

- 搜索功能仅在买家模块可用，而卖家用户理论上也应具备搜索书籍的能力。
- 初版实现逻辑较为简陋，难以满足实验要求的多功能搜索需求。

因此，在 `be/model/book.py` 中新增了优化的查找逻辑，以支持更通用和功能丰富的搜索。

7.3 搜索指定标题的图书 (`search_title`)

在 `book` 模块中，提供了按书名搜索图书的功能，包括店铺限定搜索。

7.3.1 函数说明

- `search_title`：查询全局书籍集合中指定标题的图书。
 - 参数：
 - * `title`：指定书籍标题。
 - * `page_num`：页码。
 - * `page_size`：每页显示的数量。
 - 返回值：一个元组，包含状态码、消息和结果列表。
- `search_title_in_store`：在指定商店中查询指定标题的图书。
 - 参数：
 - * `title`：指定书籍标题。
 - * `store_id`：商店 ID，可选。
 - * `page_num`：页码。
 - * `page_size`：每页显示数量。
 - 返回值：一个元组，包含状态码、消息和结果列表。

7.3.2 运行逻辑

1. 在 `book_col` 集合中查询指定 `title` 的书籍。
2. 使用 `find` 方法，并将 `_id` 字段设置为 0 以舍弃 `_id`。
3. 对查询结果执行分页处理，根据前端传入的 `page_num` 和 `page_size` 进行跳过与限制。
4. 将查询结果转换为 `list`。
5. 若结果列表为空，返回状态码 501、错误信息和空列表。
6. 若结果列表非空，返回状态码 200、消息“ok”及结果列表。

7.3.3 店铺限定逻辑 (`search_title_in_store`)

在 `search_title_in_store` 中，增加了对指定商店的筛选：

1. 遍历分页后的结果列表，获取每本书的 `id`。
2. 查询 `store_col` 集合中 `books` 数组，判断书籍是否属于指定 `store_id`。
3. 若书籍存在于指定商店，将其加入最终返回列表；否则舍弃。

7.3.4 性能优化说明

- 方法一：先查询商店中所有 `book_id`，再在 `book` 集合中逐一匹配标题。缺点是 `book_id` 不连续，随机 I/O 开销大。
- 方法二（当前实现）：先在 `book` 集合中筛选指定标题的书籍，结果集较小，再在指定商店中验证其存在性。优点是过滤大部分无关书籍，减少随机 I/O，效率更高。

7.4 搜索指定标签的图书 (`search_tag`)

在 `book` 模块中，提供了按标签搜索图书的功能，包括店铺限定搜索。

7.4.1 函数说明

- `search_tag`：查询全局书籍集合中含有指定标签的图书。
 - 参数：
 - * `tag`：指定图书标签。
 - * `page_num`：页码。
 - * `page_size`：每页显示的数量。
 - 返回值：一个元组，包含状态码、消息和结果列表。
- `search_tag_in_store`：在指定商店中查询含有指定标签的图书。

- 参数：
 - * *tag*: 指定图书标签。
 - * *store_id*: 商店 ID, 可选。
 - * *page_num*: 页码。
 - * *page_size*: 每页显示数量。
- 返回值: 一个元组, 包含状态码、消息和结果列表。

7.4.2 运行逻辑

1. 在 *book_col* 集合中查询 *tags* 数组包含指定 *tag* 的书籍。
2. 使用 *find* 方法, 并将 *_id* 字段设置为 0 以舍弃 *_id*。
3. 对查询结果执行分页处理, 根据前端传入的 *page_num* 和 *page_size* 进行跳过与限制。
4. 将查询结果转换为 *list*。
5. 若结果列表为空, 返回状态码 501、错误信息 (指定标签的图书不存在) 和空列表。
6. 若结果列表非空, 返回状态码 200、消息 "ok" 及结果列表。

7.4.3 店铺限定逻辑 (*search_content_in_store*)

在 *search_content_in_store* 中, 增加了对指定商店的筛选:

1. 遍历分页后的结果列表, 获取每本书的 *id*。
2. 查询 *store_col* 集合中 *books* 数组, 判断书籍是否属于指定 *store_id*。
3. 若书籍存在于指定商店, 将其加入最终返回列表; 否则舍弃。

7.4.4 运行优化说明

该实现逻辑与 *search_title_in_store* 和 *search_tag_in_store* 类似, 先通过全文索引过滤出满足条件的书籍, 再在指定商店中验证其存在性, 从而减少随机 I/O, 提升查询效率。

7.5 查询订单历史 (*check_hist_order*)

在 *buyer.py* 中, *check_hist_order* 函数用于查询指定用户的历史订单信息。

7.5.1 函数说明

- 参数：
 - *user_id*: 用户 ID。
- 返回值：
 - 返回历史订单信息列表 *ans*。每个元素为一个字典，包含订单状态、订单 ID、买家 ID、商店 ID、总价和书籍详细信息。
 - 若没有历史订单，则返回空列表，并附成功消息。
 - 若执行过程中出现异常，返回状态码 528 及异常信息。

7.5.2 函数主要流程

1. 检查用户是否存在，若不存在返回非存在用户 ID 的错误消息。
2. 初始化空列表 *ans* 用于存储历史订单信息。
3. 按订单状态依次查询：
 - 未付款订单 (*status* = 0)；
 - 已支付但未发货、已支付但未收到、已收到订单 (*status* = 1, 2, 3)；
 - 已取消订单 (*status* = 4)。
4. 对于每个订单状态：
 - (a) 查找匹配用户 ID 和订单状态的订单信息。
 - (b) 查询每个订单对应的书籍详细信息。
 - (c) 构建包含订单状态、订单 ID、买家 ID、商店 ID、总价及书籍信息的字典，并添加到 *ans* 列表。
5. 若未找到任何历史订单，返回成功消息及空列表；否则返回成功消息及包含历史订单信息的 *ans* 列表。
6. 捕获执行过程中可能发生的异常，返回状态码 528 及异常信息。

7.5.3 函数主要流程

1. 检查用户是否存在，若不存在返回非存在用户 ID 的错误消息。
2. 初始化空列表 *ans* 用于存储历史订单信息。
3. 按订单状态依次查询：
 - 未付款订单 (*status* = 0)；
 - 已支付但未发货、已支付但未收到、已收到订单 (*status* = 1, 2, 3)；

- 已取消订单 ($status = 4$)。

4. 对于每个订单状态：

- (a) 查找匹配用户 ID 和订单状态的订单信息。
- (b) 查询每个订单对应的书籍详细信息。
- (c) 构建包含订单状态、订单 ID、买家 ID、商店 ID、总价及书籍信息的字典，并添加到 *ans* 列表。

5. 若未找到任何历史订单，返回成功消息及空列表；否则返回成功消息及包含历史订单信息的 *ans* 列表。

6. 捕获执行过程中可能发生的异常，返回状态码 528 及异常信息。

7.6 取消订单 (*cancel_order*)

在 *buyer.py* 中，*cancel_order* 函数用于取消指定用户的订单。

7.6.1 函数说明

- 参数：
 - *user_id*: 用户 ID。
 - *order_id*: 订单 ID。
- 返回值：
 - 一个整数：操作状态码。200 表示成功，528 表示出现错误。
 - 一个字符串：描述操作结果的消息或错误信息。

7.6.2 函数主要流程

1. 首先尝试查找订单状态为 0（未付款）的订单：
 - (a) 若找到，提取订单的买家 ID、商店 ID 和订单价格。
 - (b) 从订单集合中删除该订单。
2. 若未找到未付款订单，则查找订单状态为 1、2 或 3 的订单：
 - (a) 若找到，提取订单的买家 ID、商店 ID 和订单价格。
 - (b) 检查买家 ID 是否与给定的用户 ID 匹配，若不匹配返回授权失败错误。
 - (c) 查找卖家 ID，从卖家账户中扣除订单价格，同时将金额返还给买家账户。
 - (d) 从订单集合中删除该订单。
3. 若上述步骤均未找到匹配订单，返回无效订单 ID 错误消息。
4. 查询订单详细信息集合，遍历每本书籍，将库存还原，增加相应数量。

5. 创建新订单，状态设为 4（已取消），包括订单 ID、用户 ID、商店 ID、订单价格和状态，并插入到订单集合中。
6. 捕获执行过程中可能发生的异常，返回状态码 528 及异常信息。

7.7 自动取消订单 (*auto_cancel_order*)

在 *buyer.py* 中，*auto_cancel_order* 用于自动取消长时间未支付的订单，并恢复库存。

7.7.1 函数说明

- 参数：无
- 返回值：
 - 一个整数：操作状态码。200 表示成功，528 表示出现错误。
 - 一个字符串：描述操作结果的消息或错误信息。

7.7.2 函数主要流程

1. 定义等待时间 *wait_time*（未支付订单自动取消的阈值），例如 20 秒。
2. 获取当前 UTC 时间 *now*，计算时间间隔 $interval = now - wait_time$ 。
3. 构建查询条件 *cursor*，查找订单状态为 0（未支付）且创建时间早于 *interval* 的订单。
4. 执行查询，将待取消订单信息存储在 *orders_to_cancel* 中。
5. 遍历 *orders_to_cancel*：
 - (a) 获取订单 ID、用户 ID、商店 ID 和订单价格。
 - (b) 从订单集合中删除订单。
 - (c) 查询被取消订单的书籍详细信息，遍历每本书籍：
 - i. 恢复库存，增加对应数量。
 - ii. 若库存恢复失败 ($update_result.modified_count = 0$)，返回库存不足错误。
 - (d) 构建新的订单文档 *canceled_order*，状态设为 4（已取消），并插入订单集合。
6. 捕获执行过程中可能发生的异常，返回状态码 528 及异常信息。

7.7.3 检查订单是否已取消 (*is_order_cancelled*)

该函数用于判断指定订单是否已经被取消。

- 参数：
 - *self*: 类实例引用。
 - *order_id*: 订单 ID。
- 返回值：
 - 一个整数: 操作状态码。200 表示订单已取消, 其他值表示不同错误。
 - 一个字符串: 消息或错误信息。

7.7.4 函数主要流程

1. 查询订单集合, 查找订单 ID 等于 *order_id* 且状态为 4 (已取消) 的订单。
2. 若找到符合条件的订单, 返回状态码 200 及消息 “ok”。
3. 若未找到符合条件的订单, 返回指示自动取消失败的错误消息。

8 接口与测试

8.1 接口

后端接口位于 *be/view/*, 前端接口位于 *fe/access/*。

8.1.1 后端接口

auth.py 位于 *be/view/auth.py* 中的接口未做修改, 其中:

- *login*: 登录
- *logout*: 登出
- *register*: 注册
- *unregister*: 注销
- *password*: 修改密码

buyer.py 位于 *be/view/buyer.py* 中提供了几个基本接口:

- *new_order*: 发起新订单
- *payment*: 支付
- *add_funds*: 增加余额

`seller.py` 位于 `be/view/seller.py` 中提供了几个基本接口：

- `create_store`: 创建商店
- `add_book`: 增加书目
- `add_stock_level`: 增加库存

8.1.2 buyer 和 seller 新增功能接口

`buyer.py` 我们新增了以下接口：

- `receive_books`: 收书
- `cancel_order`: 取消订单
- `auto_cancel_order`: 自动取消订单
- `check_order_is_cancelled`: 检验订单是否已被取消
- `check_history_order`: 查询订单历史
- `search`: 简单搜索

`seller.py` 我们新增了一个接口：

- `hand_out_books`: 发书功能

8.1.3 精细化搜索功能接口

位于 `be/view/search.py`。

实现流程 基本流程如下：

1. 接口路由为 `"/search/tag_in_store"`，请求方法为 GET。
2. 使用 Flask 内置对象 `request` 获取请求参数：
 - `title`: 图书标题
 - `page_num`: 页号
 - `page_size`: 每页条数
3. 检查请求参数是否为 None，若为 None 则赋予默认值。
4. 调用 `be.model.book` 中对应的方法进行查询。
5. 将返回值封装成字典，并调用 `jsonify` 进行序列化，返回给前端。

返回数据结构 搜索接口返回数据通常为字典结构，包含：

- 状态码 (status_code)
- 消息 (message)
- 搜索结果列表 (data)

"data": books, "message": message, "code": code 我们以此为模板实现了对应的搜索指定标题的图书接口
(search_title tag search_tag content search_content author search_author ,tag,content,author

8.2 前端接口

在修改完后端接口后，我们在 fe/access 实现了对应的前端接口，流程如下：

1. 拼接得到对应的接口路由。
2. 使用 requests 库的 get 方法发送 GET 请求，params 指定请求参数。
3. 获取 response 后，调用 json.loads 方法解析返回的 JSON 字符串。
4. 返回执行码，即 JSON 对象中的 code，以及对应的 message 和 data。

返回的 JSON 数据结构如下：

```
{
  "data": books,
  "message": message,
  "code": code
}
```

8.3 测试

在 fe/test 文件夹下，除了原有的 33 个基础测试点，我们还为新增功能添加了大量额外测试。新测试类的初始化流程为：

1. 使用 uuid 构造唯一标识，例如 seller_id。
2. 构造 password。
3. 调用相应接口进行注册。
4. 注册成功后执行相关操作，如创建店铺。

8.3.1 test_cancel_order

该测试类包含以下具体测试：

1. **test_paid 情况：**已付款的订单。**操作：**创建订单，付款，取消订单。**预期结果：**取消订单成功。
2. **test_unpaid 情况：**未付款的订单。**操作：**创建订单，取消订单。**预期结果：**取消订单成功。
3. **test_invalid_order_id_paid 情况：**已付款订单，使用不存在的订单 ID 取消。**操作：**创建订单，付款，使用不存在的订单 ID 取消订单。**预期结果：**取消订单失败。
4. **test_invalid_order_id_unpaid 情况：**未付款订单，使用不存在的订单 ID 取消。**操作：**创建订单，使用不存在的订单 ID 取消订单。**预期结果：**取消订单失败。
5. **test_authorization_error_paid 情况：**已付款订单，用户 ID 不存在。**操作：**创建订单，付款，使用不存在的用户 ID 取消订单。**预期结果：**取消订单失败。
6. **test_authorization_error_unpaid 情况：**未付款订单，用户 ID 不存在。**操作：**创建订单，使用不存在的用户 ID 取消订单。**预期结果：**取消订单失败。
7. **test_repeat_cancel_paid 情况：**已付款订单，尝试重复取消。**操作：**创建订单，付款，取消订单，再次取消订单。**预期结果：**第一次取消成功，第二次取消失败。
8. **test_repeat_cancel_not_paid 情况：**未付款订单，尝试重复取消。**操作：**创建订单，取消订单，再次取消订单。**预期结果：**第一次取消成功，第二次取消失败。

8.3.2 test_cancel_auto

该测试类主要测试自动取消订单的不同情况，具体测试点如下：

1. **test_overtime 情况：**订单在规定时间内未付款，自动取消。**操作：**创建订单，等待超过规定时间，检查订单是否被取消。**预期结果：**订单被取消，返回码为 200。
2. **test_overtime_paid 情况：**订单在规定时间内已付款，超时后检查订单状态。**操作：**创建订单，付款，等待超过规定时间，检查订单是否被取消。**预期结果：**订单已付款，不应该被取消，返回码不为 200。
3. **test_overtime_canceled_by_buyer 情况：**订单在规定时间内被买家取消，超时后检查订单状态。**操作：**创建订单，买家取消订单，等待超过规定时间，检查订单是否被取消。**预期结果：**订单已被买家取消，应被取消，返回码为 200。

8.3.3 test_history_order

该测试类主要测试查询历史订单功能：

1. **test_have_orders 情况：**用户有历史订单。**操作：**创建 10 个订单（可能包含取消、已付款、发货和收货的情况），调用 check_hist_order。**预期结果：**返回码为 200。

2. **test_non_exist_user_id 情况:** 使用不存在的用户 ID 查询历史订单。**操作:** 调用 check_hist_order。**预期结果:** 返回码不为 200。

3. **test_no_orders 情况:** 用户没有历史订单。**操作:** 调用 check_hist_order。**预期结果:** 返回码为 200。

8.3.4 test_receive

该测试类主要测试买家收货功能:

1. **test_ok 情况:** 正常收货。**操作:** 卖家发货, 买家收货。**预期结果:** 收货成功, 返回码为 200。

2. **test_order_error 情况:** 订单 ID 不存在。**操作:** 卖家发货, 使用不存在的订单 ID 尝试收货。**预期结果:** 收货失败, 返回码不为 200。

8.3.5 test_send

该测试类主要测试卖家发货功能:

1. **test_ok 情况:** 正常发货。**操作:** 卖家发货。**预期结果:** 发货成功, 返回码为 200。

2. **test_order_error 情况:** 订单 ID 不存在。**操作:** 使用不存在的订单 ID 尝试发货。**预期结果:** 发货失败, 返回码不为 200。

3. **test_authorization_error 情况:** 使用不存在的卖家 ID 尝试发货。**操作:** 使用不存在的卖家 ID 尝试发货。**预期结果:** 发货失败, 返回码不为 200。

4. **test_books_repeat_send 情况:** 重复发货。**操作:** 卖家发货, 再次尝试发货。**预期结果:** 第一次发货成功, 第二次发货失败, 返回码不为 200。

8.3.6 test_search

该测试类主要测试搜索功能, 包括全属性搜索、分页搜索以及按标题、标签、内容、作者搜索等功能:

1. **test_all_field_search** 调用 buyer 的 search 方法查询指定 keyword 的图书。**预期结果:** keyword 存在且接口逻辑正确时, 返回码为 200。

2. **test_pagination** 调用 buyer 的 search 方法进行分页查询。**预期结果:** 返回码为 200。

3. **test_search_title** 构造数据库中不存在的 title 的图书, 调用 seller 的 add_book 方法加入此图书。调用 request_search_title 方法查询已存在的 title 和不存在的 title。**预期结果:** 已存在的 title 返回码为 200, 不存在的 title 返回码为 501。

4. **test_search_title_in_store** 在指定 store 中搜索指定 title 的图书。**预期结果:** 已存在的 title 返回码为 200, 不存在的 title 返回码为 501。

5. **test_search_tag** 根据 tag 搜索图书。**预期结果:** 已存在的 tag 返回码为 200, 不存在的 tag 返回码为 501。

6. `test_search_tag_in_store` 在指定 store 中根据 tag 搜索图书。**预期结果：**已存在的 tag 返回码为 200，不存在的 tag 返回码为 501。
7. `test_search_content` 根据 content 搜索图书。**预期结果：**已存在的 content 返回码为 200，不存在的 content 返回码为 501。
8. `test_search_content_in_store` 在指定 store 中根据 content 搜索图书。**预期结果：**已存在的 content 返回码为 200，不存在的 content 返回码为 501。
9. `test_search_author` 根据 author 搜索图书。**预期结果：**已存在的 author 返回码为 200，不存在的 author 返回码为 501。
10. `test_search_author_in_store` 在指定 store 中根据 author 搜索图书。**预期结果：**已存在的 author 返回码为 200，不存在的 author 返回码为 501。

9 测试模块总结（表格版）

9.1 测试概述

在 `fe/test` 文件夹下，除原有 33 个基础测试外，我们新增了多项扩展测试，涵盖订单取消、自动超时取消、历史订单查询、发货收货流程及搜索功能。

所有测试类的初始化流程如下：

1. 使用 `uuid` 生成唯一标识（如 `seller_id`）。
2. 构造 `password`。
3. 调用注册接口。
4. 注册成功后创建商店等初始化操作。

总结：所有测试均覆盖买家与卖家主要

10 遇到的主要问题展示

10.1 store 的 books 插入过多信息导致 MongoDB 限制

最初设计中，一个 store 对应多本书，store 中存放一个 books 列表，列表的每一个元素都包含如下信息：

- `book_id`
- `book_info`
- `stock_level`

但是随着书籍数量的增加，MongoDB 报错如下：

BSONObj size: 17020874 (0x103B7CA) is invalid. Size must be between 0 and 16793600(16MB)

原因是 MongoDB 文档的默认大小上限为 16MB。由于 books 中的 book_info 信息较大，导致 store 文档超出限制。最终导致 test_bench 无法通过，而其他测试均能通过。

解决方案：

- 将 store 中的 book_info 字段移除，仅保留 book_id 和 stock_level。
- 需要获取具体书籍信息时，直接从 book_col 集合中查找即可。

10.2 fe/access/book.py 修改问题

最初我们将 fe/access/book.py 改为使用 MongoDB 操作，将本地 book.db 导入到 MongoDB 中。修改后，除了 test_bench 外，其他测试均能通过。

后来查阅水杉论坛助教学长的说明：

- **SQL 操作无需修改为 MongoDB 操作**，因为 fe/access/book.py 仅用于读取本地 book.db 或 book_lx.db，并调用 add_book 插入自定义数据库，仅用于性能测试。
- 如果已经改为 MongoDB 操作并且 test_bench 能通过，则无需回退为 SQL。

最终方案：

- 将 fe/access/book.py 替换为原始版本，其他代码不修改，test_bench 顺利通过。

11 实验分工及结果

11.1 小组分工

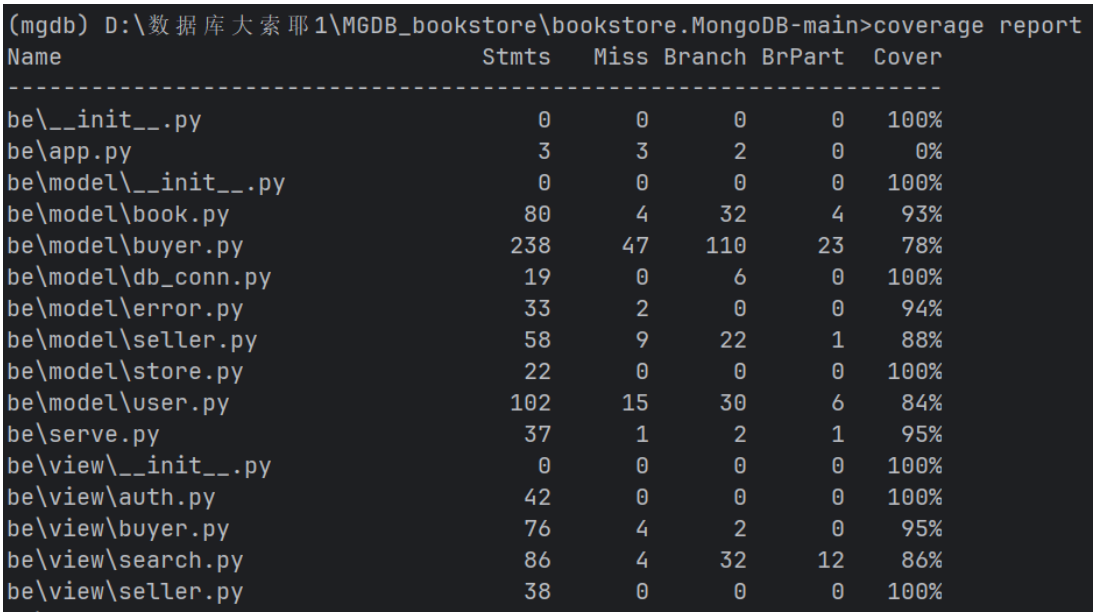
- **罗文琦**：基本用户功能实现、基本买家功能实现、数据库模式设计、手动及自动取消订单、查询历史订单、收货与发货、实验报告撰写。
- **孙嘉**：git 版本控制、基本卖家功能实现、搜索功能（含优化及各种情况）、前端与后端接口测试、bug 调试、数据库性能提升、实验报告撰写。

11.2 测试结果

- 基础功能通过测试。
- 遇到 MongoDB 文档大小限制时，出现报错：

An error occurred: BSONObj size: 17020874 (0x103B7CA) is invalid. Size must be between 0 and 16793600(16MB) First element: __id: ObjectId('65499f4c6ead219cc4eea59b')

12 实验结果



```
(mgdb) D:\数据库大索耶1\MGDB_bookstore\bookstore.MongoDB-main>coverage report
```

Name	Stmts	Miss	Branch	BrPart	Cover
be__init__.py	0	0	0	0	100%
be\app.py	3	3	2	0	0%
be\model__init__.py	0	0	0	0	100%
be\model\book.py	80	4	32	4	93%
be\model\buyer.py	238	47	110	23	78%
be\model\db_conn.py	19	0	6	0	100%
be\model\error.py	33	2	0	0	94%
be\model\seller.py	58	9	22	1	88%
be\model\store.py	22	0	0	0	100%
be\model\user.py	102	15	30	6	84%
be\serve.py	37	1	2	1	95%
be\view__init__.py	0	0	0	0	100%
be\view\auth.py	42	0	0	0	100%
be\view\buyer.py	76	4	2	0	95%
be\view\search.py	86	4	32	12	86%
be\view\seller.py	38	0	0	0	100%

图 1: 图片说明文字

fe__init__.py	0	0	0	0	100%
fe\access__init__.py	0	0	0	0	100%
fe\access\auth.py	31	0	0	0	100%
fe\access\book.py	70	1	12	2	96%
fe\access\buyer.py	73	6	4	1	91%
fe\access\new_buyer.py	8	0	0	0	100%
fe\access\new_seller.py	8	0	0	0	100%
fe\access\search.py	53	0	0	0	100%
fe\access\seller.py	37	0	0	0	100%
fe\bench__init__.py	0	0	0	0	100%
fe\bench\run.py	13	0	6	0	100%
fe\bench\session.py	47	0	12	1	98%
fe\bench\workload.py	125	2	22	2	97%
fe\conf.py	11	0	0	0	100%
fe\conftest.py	17	0	0	0	100%
fe\test\gen_book_data.py	49	1	16	2	95%
fe\test\test_add_book.py	36	0	10	0	100%
fe\test\test_add_funds.py	23	0	0	0	100%
fe\test\test_add_stock_level.py	39	0	10	0	100%
fe\test\test_bench.py	7	3	0	0	57%
fe\test\test_cancel_auto.py	54	1	4	1	97%
fe\test\test_cancel_order.py	82	1	4	1	98%
fe\test\test_create_store.py	20	0	0	0	100%
fe\test\test_history_order.py	65	1	14	2	96%
fe\test\test_login.py	28	0	0	0	100%
fe\test\test_new_order.py	40	0	0	0	100%
fe\test\test_password.py	33	0	0	0	100%
fe\test\test_payment.py	60	1	4	1	97%
fe\test\test_receive.py	58	1	4	1	97%
fe\test\test_register.py	31	0	0	0	100%
fe\test\test_search.py	98	0	0	0	100%
fe\test\test_send.py	47	1	4	1	96%

TOTAL	2097	108	364	62	93%

图 2: 图片说明文字

```
终端 本地 Command Prompt x + v
2025-10-30 10:28:46,514 [Thread-11901] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:46] "POST /seller/add_book HTTP/1.1" 200 -
2025-10-30 10:28:46,561 [Thread-11902] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:46] "POST /seller/add_book HTTP/1.1" 200 -
2025-10-30 10:28:46,605 [Thread-11903] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:46] "POST /auth/register HTTP/1.1" 200 -
2025-10-30 10:28:46,655 [Thread-11904] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:46] "POST /auth/login HTTP/1.1" 200 -
2025-10-30 10:28:46,696 [Thread-11905] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:46] "POST /buyer/add_funds HTTP/1.1" 200 -
2025-10-30 10:28:46,795 [Thread-11906] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:46] "POST /buyer/new_order HTTP/1.1" 200 -
2025-10-30 10:28:46,844 [Thread-11907] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:46] "POST /buyer/payment HTTP/1.1" 200 -
2025-10-30 10:28:46,901 [Thread-11908] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:46] "POST /seller/send_books HTTP/1.1" 518 -
PASSED [ 96%]
fe/test/test_send.py::TestSend::test_authorization_error 2025-10-30 10:28:46,938 [Thread-11909] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:46] "POST /auth/register HTTP/1.1" 200 -
2025-10-30 10:28:46,980 [Thread-11910] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:46] "POST /auth/login HTTP/1.1" 200 -
2025-10-30 10:28:47,009 [Thread-11911] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /seller/create_store HTTP/1.1" 200 -
2025-10-30 10:28:47,077 [Thread-11912] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /seller/add_book HTTP/1.1" 200 -
2025-10-30 10:28:47,118 [Thread-11913] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /auth/register HTTP/1.1" 200 -
2025-10-30 10:28:47,167 [Thread-11914] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /auth/login HTTP/1.1" 200 -
2025-10-30 10:28:47,214 [Thread-11915] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /buyer/add_funds HTTP/1.1" 200 -
2025-10-30 10:28:47,284 [Thread-11916] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /buyer/new_order HTTP/1.1" 200 -
2025-10-30 10:28:47,346 [Thread-11917] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /buyer/payment HTTP/1.1" 200 -
2025-10-30 10:28:47,389 [Thread-11918] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /seller/send_books HTTP/1.1" 481 -
PASSED [ 99%]
fe/test/test_send.py::TestSend::test_books_repeat_send 2025-10-30 10:28:47,432 [Thread-11919] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /auth/register HTTP/1.1" 200 -
2025-10-30 10:28:47,495 [Thread-11920] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /auth/login HTTP/1.1" 200 -
2025-10-30 10:28:47,547 [Thread-11921] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /seller/create_store HTTP/1.1" 200 -
2025-10-30 10:28:47,613 [Thread-11922] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /seller/add_book HTTP/1.1" 200 -
2025-10-30 10:28:47,677 [Thread-11923] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /seller/add_book HTTP/1.1" 200 -
2025-10-30 10:28:47,742 [Thread-11924] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /seller/add_book HTTP/1.1" 200 -
2025-10-30 10:28:47,778 [Thread-11925] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /seller/add_book HTTP/1.1" 200 -
2025-10-30 10:28:47,837 [Thread-11926] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /auth/register HTTP/1.1" 200 -
2025-10-30 10:28:47,889 [Thread-11927] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /auth/login HTTP/1.1" 200 -
2025-10-30 10:28:47,936 [Thread-11928] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:47] "POST /buyer/add_funds HTTP/1.1" 200 -
2025-10-30 10:28:48,056 [Thread-11929] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:48] "POST /buyer/new_order HTTP/1.1" 200 -
2025-10-30 10:28:48,121 [Thread-11930] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:48] "POST /buyer/payment HTTP/1.1" 200 -
2025-10-30 10:28:48,154 [Thread-11931] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:48] "POST /seller/send_books HTTP/1.1" 200 -
2025-10-30 10:28:48,175 [Thread-11932] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:48] "POST /seller/send_books HTTP/1.1" 522 -
PASSED [100%]
===== 66 passed in 739.71s (0:12:19) =====
D:\数据库大课第1\MOODB_bookstore\bookstore.MongDB-main\be\serve.py:19: UserWarning: The 'environ'['werkzeug.server.shutdown'] function is deprecated and will be removed in Werkzeug 2.1.
func()
2025-10-30 10:28:48,191 [Thread-11933] [INFO] 127.0.0.1 - - [30/Oct/2025 10:28:48] "GET /shutdown HTTP/1.1" 200 -
frontend end test
2025-10-30 10:28:49,184 [APScheduler] [ERROR] Error submitting job "Buyer auto cancel order (trigger: interval[0:00:05], next run at: 2025-10-30 10:28:49 CST)" to executor "default"
```

图 3: 图片说明文字

13 实验总结

14 附录

本实验的实验仓库如下：